These aspects allow us to illustrate an interesting technique called *memoization*, which can be used, in specific contexts, to improve the performance of recursive algorithms.

The same results could be obtained analogously in most functional programming languages. There exist workarounds in OO languages too: for instance, in Java, memoization can be obtained through static class fields.

E.1 Simple Recursion

The simplest case of recursion is when a function calls itself at a given point of its flow.

The most used (and abused) examples of recursion are computing the factorial of a number, or *Fibonacci*'s numbers.

Here it is a possible JavaScript implementation of a function computing Fibonacci's numbers.

Listing E.1 Fibonacci Numbers, JavaScript

```
function fibonacci(n) {
     if (n < 0) {
      throw new RangeError('n can not be negative');
     } else if (n === 0 || n === 1) {
       return 1; // Base case
     } else {
       return fibonacci(n-1) + fibonacci(n-2);
     }
}
```